

Cluster failures, libraries and memory

Why clusters do not start, why libraries fight each other, and which component actually ran out of memory.

Ask **when** it failed. Before the VMs launched, during startup, or while running. Those three points have almost no causes in common.

Three moments, three families

WHEN IT FAILED

The VMs never launched

The cluster never became ready

The job failed while running

WHAT TO LOOK AT

Cloud quota, capacity, subnet, IP addresses

Init scripts, libraries, bootstrap timeouts

Memory, skew, spill — and which component

ON THE EXAM

The termination reason on the cluster event log tells you which row you are in. Read it before doing anything else.

When the cloud provider refuses

- ◆ **Quota exceeded.** The launch would take the account past a limit on instances, cores, volumes or IP addresses.
- ◆ **Stockout.** The availability zone has no capacity for that instance type right now.
- ◆ **A full subnet, or no free public IPs.** Both are common and both look like a launch failure.
- ◆ **The reason is `CLOUD_PROVIDER_LAUNCH_FAILURE`,** and the parameters carry the provider's own error code.

ON THE EXAM

None of these are fixed by changing your Databricks configuration. They are fixed in the cloud account, or by asking for different machines.

Fixing a launch failure

- ◆ **For quota:** request an increase, terminate unused clusters, or use different instance types.
- ◆ **For a stockout:** enable flexible node types, try another availability zone, or choose alternative instances.
- ◆ **Check compute policies allow the fallback,** or your alternative instance type is blocked before it is tried.
- ◆ **Intermittent launch failures are often transient,** and re-running succeeds. Persistent ones are not.

ON THE EXAM

Terminating idle clusters is the fastest quota fix, and module 2's auto-termination is what stops the problem recurring.

Init script failures

- ◆ Init scripts run on the driver and on every worker node.
- ◆ A failure on any single node can terminate the whole cluster.
- ◆ Cluster-scoped failures surface as `INIT_SCRIPT_FAILURE`.
- ◆ Admin-configured global init scripts have their own reason, and usually need an admin to fix.

ON THE EXAM

A script that works on your laptop and on the driver can still fail on a worker. Every node runs it, so every node must succeed.

Reading an init script failure

- ◆ **Read the init script logs** to find which script failed and why.
- ◆ **If it could not be fetched**, check the path still exists and the identity can read it.
- ◆ **If it ran and exited non-zero**, find the failing command in its output.
- ◆ **If it installs packages, check the repositories are reachable** from the cluster's network.

ON THE EXAM

Enable init script logging before you need it. Diagnosing a script with no logs means reproducing the failure to see it.

Startup timeouts

- ◆ **VM bootstrap has a 700-second timeout**, and can be exceeded by network problems or slow artifact downloads.
- ◆ **The Spark driver must become ready within a few minutes**, or the launch fails.
- ◆ **Too many libraries configured in the cluster UI** can push it past that window.
- ◆ **Library installation has its own timeout**, and JAR fetching over a slow network is a common cause.

ON THE EXAM

A cluster connected to an external Hive metastore can download around 200 JARs at startup, which alone can exceed the driver-ready window.

The Databricks Runtime already contains hundreds of libraries. A conflict is usually **your version fighting the one that was already there.**

Cluster libraries or notebook-scoped?

CLUSTER-SCOPED

Installed on the cluster

- Available to every notebook and job on that cluster
- Installed at cluster startup, before it is usable
- A bad one can prevent the cluster starting
- Right for dependencies the whole team needs

NOTEBOOK-SCOPED

%pip install in the notebook

- Scoped to that notebook's session
- Installed when the cell runs, not at startup
- A bad one fails the cell, not the cluster
- Right for experimenting, and for per-job dependencies

THE TIEBREAKER

Notebook-scoped isolates the blast radius. A broken cluster library affects everyone; a broken %pip affects one notebook.

The classic conflict

- ◆ **Installing your own NumPy or pandas over the runtime's version** can break the runtime's compiled extensions.
- ◆ **The symptom is often not a package error.** The driver crashes on startup, and you see a 'could not reach driver' message.
- ◆ **The fix is to remove the custom versions** and use the ones that ship with the runtime.
- ◆ **Or move to a runtime version that ships the version you need**, rather than overriding it.

ON THE EXAM

'Could not reach driver' sounds like a network problem and is frequently a library problem. That mismatch is what makes it hard.

Libraries and access mode

- ◆ On standard access mode, libraries and init scripts must be on the Unity Catalog allowlist.
- ◆ The allowlist is empty by default, and modifying it requires the `MANAGE ALLOWLIST` privilege.
- ◆ Libraries used as JDBC drivers or custom data sources need **ANY FILE** permission on standard access mode.
- ◆ **Identity differs by mode:** on standard access mode, libraries use the installer's identity and init scripts the cluster owner's.

ON THE EXAM

A library that installs on a dedicated cluster and fails on a standard one is usually the allowlist, not the library.

The first question is never how much memory. It is **which component ran out** — because driver and executor OOM have almost nothing in common.

Driver out of memory

- ◆ **collect and toPandas** pull the entire result back to the driver.
- ◆ **An oversized broadcast**, because the small side is collected on the driver before distribution.
- ◆ **Very large query plans**, or a loop building thousands of DataFrames.
- ◆ **Adding executors does nothing.** The driver is one machine and more workers do not help it.

ON THE EXAM

The driver does not process partitions. So dataset size alone does not pressure it — something has to pull data to it.

Executor out of memory

- ◆ **Partitions too large to process in the memory each task has.**
- ◆ **Skew**, where one partition is enormous while the rest are fine.
- ◆ **A wide row or a large object held per task**, such as a model loaded inside a UDF.
- ◆ **Spill usually comes first.** An executor that is spilling heavily is on its way to an OOM.

ON THE EXAM

Module 24's ordering applies: more partitions before more memory, because the partition is what did not fit.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
The VMs never launched	Cloud quota, capacity, subnet or IP addresses
The cluster started then terminated during startup	An init script — it runs on every node
A global init script failed	An admin must fix it, not the cluster's user
'Could not reach driver' after a library change	A library conflicting with the runtime's version
A library works on dedicated but not standard	The Unity Catalog allowlist
The error names the driver	collect, toPandas, or an oversized broadcast
The error names an executor	Partition size or skew, before memory

ON THE EXAM

Seven lines. The first two are the split that saves the most time, and the sixth and seventh are the split that saves the most money.

A team's scheduled jobs begin failing intermittently each morning between 08:00 and 09:30, when several teams start work. The cluster event log shows `CLOUD_PROVIDER_LAUNCH_FAILURE` with a provider error code about an address limit. Re-running the same job at 11:00 works every time.

What is happening?

- ☐ A A code defect that only manifests under morning data volumes
- ☐ B A cloud account limit is being reached at peak, most likely public IP addresses consumed by concurrently running clusters
- ☐ C An init script is failing during the busy period
- ☐ D The jobs need a longer timeout configured

◆ PAUSE AND DECIDE

What is happening?

- A

A code defect

The job never started. The cluster failed to launch, so no code of yours has run at all.
- B

A cloud account limit reached at peak

Correct. The reason names the cloud provider and an address limit, and the timing tracks how many clusters are running concurrently.
- C

An init script failing

That produces INIT_SCRIPT_FAILURE, and it would mean the VMs launched. The reason here says they did not.
- D

A longer timeout

Nothing timed out. The provider refused the request outright, and waiting longer does not create IP addresses.

THE SIGNAL

The termination reason tells you the row before you reason at all.
CLOUD_PROVIDER_LAUNCH_FAILURE means nothing of yours ever ran.

An engineer adds a cluster-scoped init script that installs a package from an internal repository. On his own 2-node development cluster it works every time.

On the 40-node production cluster the launch fails with `INIT_SCRIPT_FAILURE`, but not on every attempt — roughly one launch in three succeeds.

What is the most likely explanation?

- ☐ A The production cluster has an incompatible runtime version
- ☐ B Init scripts run on every node, so a transient failure reaching the repository on any one of 40 nodes terminates the cluster
- ☐ C Init scripts are not supported on clusters above a certain size
- ☐ D The script needs to be converted to a global init script

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A

An incompatible runtime version

A version incompatibility would fail consistently, not one launch in three. Intermittency points at something probabilistic.
- B

Every node runs it; one failure of 40 kills the cluster

Correct. A per-node success rate that is fine on 2 nodes becomes unreliable on 40, because all 40 must succeed.
- C

Unsupported above a size

Invented limit. Init scripts run on clusters of any size.
- D

Make it a global init script

Changes who manages it and applies it to every cluster in the workspace. It does not change the per-node execution that is failing.

THE SIGNAL

Two nodes and forty nodes are not the same test. Twenty times the nodes is twenty times the chances for one of them to fail.

A data scientist adds pandas and NumPy at specific versions as cluster-scoped libraries, because a modelling package she needs requires them. The cluster now fails to start, reporting that the driver could not be reached.

The team spends a day checking subnets, security groups and quotas. All are fine, and the cluster started normally the previous week.

What should they check?

- ☐ A Whether the newly added libraries conflict with the versions the Databricks Runtime ships
- ☐ B Whether the workspace has run out of public IP addresses
- ☐ C Whether the cluster needs a larger driver node
- ☐ D Whether an init script is failing on the worker nodes

◆ PAUSE AND DECIDE

What should they check?

- A

A conflict with the runtime's own versions

Correct. Overriding fundamental packages can break the runtime's compiled extensions, crashing the driver at startup. The message misleads.
- B

Public IP addresses

That produces `CLOUD_PROVIDER_LAUNCH_FAILURE` before any driver exists. Here the machines launched and the driver crashed.
- C

A larger driver node

Nothing suggests memory pressure. The driver did not run out of memory; it failed to come up at all.
- D

An init script on the workers

That surfaces as `INIT_SCRIPT_FAILURE`, and no init script was added. The change made was to cluster libraries.

THE SIGNAL

'Could not reach driver' reads as a network symptom, which is why a day went into networking. The change log said libraries.

A nightly job fails with an out-of-memory error. The team doubles the worker node memory. It fails again. They double it once more, and it still fails, now at considerable cost.

Reading the stack trace properly, they find it names the driver. The final step of the job calls `toPandas()` on the result to write a formatted Excel report.

What should they do?

- ☐ A Continue increasing worker memory until it succeeds
- ☐ B The failure is on the driver; `toPandas` pulls the whole result to it. Aggregate first, or write the file differently.
- ☐ C Increase `spark.sql.shuffle.partitions` to reduce partition size
- ☐ D Enable AQE to handle the skew

◆ PAUSE AND DECIDE

What should they do?

- A

 Keep increasing worker memory
Two doublings have already failed, which is the evidence that workers are not the constraint. A third will fail identically, more expensively.
- C

 More shuffle partitions
Shuffle partitions govern executor-side partition size. The driver does not process partitions, so this changes nothing for it.
- B

 It is the driver; toPandas pulls everything to it
Correct. The trace names the driver, and toPandas is the operation bringing the full result to a single machine.
- D

 Enable AQE for skew
AQE optimises distributed execution. It has no effect on a single-machine collection to the driver.

THE SIGNAL

Two failed doublings were already the answer. If more executor memory does not help, the executors were never the problem.

Read the failure before you fix it. **When did it fail, which component named the error, and what changed since it last worked.**